Implementing PCA with Scikit-learn 🖥️

Scikit-learn makes applying PCA straightforward. Here are the key steps and code examples.

1. Pre-processing: Feature Scaling

Before running PCA, it is critical to ensure all features are on a comparable scale.

- **Why?** If feature x_1 ranges from 0-1000 and x_2 ranges from 0-1, PCA will be biased toward x_1.
- **Action:** Standardize your data (e.g., using `StandardScaler` in scikit-learn) so each feature has a mean of 0 and variance of 1.

2. Fitting the Data

You use the `.fit()` method to calculate the new axes (principal components).

- **Note:** The `.fit()` function in PCA automatically performs mean normalization (centering the data), so you don't need to do that step manually (though feature scaling is still needed).

3. Inspecting Variance

After fitting, check how much information is retained using `.explained_variance_ratio_`.

- **Example:** `[0.992]` means the first component captures 99.2% of the information.
- This helps you decide if reducing dimensions (e.g., 2D to 1D) loses too much data.

4. Transforming the Data

Finally, use `.transform()` to project your original data onto the new axes. This gives you the new, compressed dataset.

Code Example

```
from sklearn.decomposition import PCA

# X is your dataset (e.g., 6 examples with 2 features)

# 1. Initialize PCA to find 1 principal component
pca_1 = PCA(n_components=1)

# 2. Fit the model to the data
pca_1.fit(X)

# 3. Check Explained Variance
```

```
# Result might be [0.992], meaning 99.2% of variance is captured
print(pca_1.explained_variance_ratio_)

# 4. Transform data to new dimensions
X_trans = pca_1.transform(X)
# X_trans is now an array of single numbers (1D data)
```

Advice on Applying PCA 💡

Primary Use Case: Visualization

- **Status: Highly Recommended / Common**
- **Purpose:** Taking high-dimensional data (50+ features) and reducing it to 2 or 3 dimensions to plot it. This helps you spot clusters, outliers, and patterns.

Secondary Use Case: Data Compression

- **Status:** *Less Common Today*
- **Purpose:** Reducing storage space or transmission bandwidth (e.g., compressing 50 features to 10).
- **Why Less Common?** Modern storage and bandwidth are cheap and plentiful, so aggressive compression is rarely necessary for structured data.

Secondary Use Case: Speeding Up Training

- **Status:** *Less Common Today*
- **Purpose:** Reducing features (e.g., 1000 $\rightarrow$ 100) to make supervised learning algorithms run faster.
- **Why Less Common?** Modern algorithms (like Neural Networks) handle high-dimensional data very well. The computational cost of running PCA often outweighs the small speedup in training.

Summary of Course 3, Week 2

This concludes the week on Recommender Systems and Dimensionality Reduction!

- **Collaborative Filtering:** Recommendations based on user behavior.
- **Content-Based Filtering:** Recommendations based on user/item features.
- **PCA:** Reducing dimensions to visualize complex data.